



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ_____

КАФЕДРА _____СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ_____

Лабораторная работа №3
По курсу
«Методы машинного обучения в АСОИУ»
«Методы многомерного поиска»

Студент ИУ5-24М
(Группа)

(Подпись, дата) Сорокин А.Е.
(И.О.Фамилия)

Проверила

(Подпись, дата) Гапанюк Ю.Е.
(И.О.Фамилия)

Цель работы

- Изучение алгоритмов многомерного поиска 1-го и 2-го порядка.
- Разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка.
- Вычисление экстремумов функции.

Задание

Требуется найти минимум тестовой функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

Методами сопряженных градиентов (методом Флетчера-Ривза и методом Полака-Рибьера).

Квазиньютоновскими методами (DFP, BFGS, и L-BFGS).

Вариант 8:

$a=158$, $b=2$, $f_0=40$, $n=2$

1. Найти все стационарные точки и значения функций, соответствующие этим точкам.
2. Оценить скорость сходимости указанных алгоритмов и сравнить по времени получение результата оптимизации для разных методов.
3. Реализовать алгоритмы программированием на одном из языков высокого уровня (C++, C#, Python, Haskell, и др.).
4. Отчет предоставить в стандартном виде (TEX, PDF).

Требования к отчету:

1. Отчет должен содержать:
 - 1.1. Титульный лист.
 - 1.2. Цель работы.
 - 1.3. Постановку задачи.
 - 1.4. Проверку решения на допустимость.
2. Исследование выполнить с помощью написанной Вами программы с результатами в графическом виде.
3. Кроме текста исследования следует привести также текст исходного кода программ.

4. Отчет оформляется в формате PDF желательно в редакторе TEX.

Найти все стационарные точки и значения функций, соответствующие этим точкам.

$$f(x) = \sum_1^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

Сначала подставим значения параметров по варианту в формулу Розеброка и получим:

$$f(x) = \sum_1^{2-1} \left[158(x_i^2 - x_{i+1})^2 + 2(x_i - 1)^2 \right] + 40 = 158(x_1^2 - x_2)^2 + \\ + 2(x_1 - 1)^2 + 40$$

Теперь необходимо вычислить градиент.

Найдем частную производную по первой переменной x_1 :

$$\frac{\partial f}{\partial x_1} = 4ax_1(x_1^2 - x_2) + 2b(x_1 - 1)$$

Теперь найдем частную производную по второй переменной x_2 :

$$\frac{\partial f}{\partial x_2} = -2a(x_1^2 - x_2)$$

Подставляем значения заданных параметров по варианту:

$$\frac{\partial f}{\partial x_1} = 632x_1(x_1^2 - x_2) + 4(x_1 - 1)$$

$$\frac{\partial f}{\partial x_2} = -316(x_1^2 - x_2)$$

Выражение:

$$\frac{\partial f}{\partial x_2} = -316(x_1^2 - x_2)$$

Принимает значение равное 0 только в случае, если: $(x_1^2 - x_2) = 0 \rightarrow x_2 = x_1^2$

Подставим это решение в первую частную производную:

$$\frac{\partial f}{\partial x_1} = 632x_1(x_1^2 - x_2) + 4(x_1 - 1) = 632x_1(x_1^2 - x_1^2) + 4(x_1 - 1) = \\ = 4(x_1 - 1) \rightarrow x_1 = 1$$

Получаем точку минимума $x_1 = 1$ и $x_2 = 1$

Подставляем найденные значения в исходную функцию:

$$f(x_1, x_2) = 158(1^2 - 1)^2 + 2(1 - 1)^2 + 40 = 40$$

Получается, что глобальный минимум исходной функции равен 40 в точке (1,1)

Проверим найденное решение посредством программирования различных алгоритмов: Fletcher-Reeves, Polak-Ribiere, BFGS, DFP, L-BFGS.

Результаты выполнения алгоритмов:

	Метод	x	f(x)	Итерации
0	Fletcher-Reeves	[1.0000000037633409, 1.00000000606340727]	40.0	68
1	Polak-Ribiere	[1.00000000888378342, 1.00000001805469871]	40.0	139
2	BFGS	[1.00000000001097906, 1.00000000002605771]	40.0	20
3	DFP	[1.00000000004033058, 1.00000000008182717]	40.0	89
4	L-BFGS	[0.99999999563490787, 0.99999999276403198]	40.0	35

	Время (сек)
0	0.013265
1	0.041054
2	0.006305
3	0.025950
4	0.094980

Как видно, все алгоритмы справились с задачей и нашли глобальный минимум для функции в точке 1,1 – это значение 40.

Разница лишь в количестве итераций в затраченном времени алгоритмами.

Дольше всех выполнялся алгоритм L-BFGS.

Оценить скорость сходимости указанных алгоритмов и сравнить по времени получение результата оптимизации для разных методов.

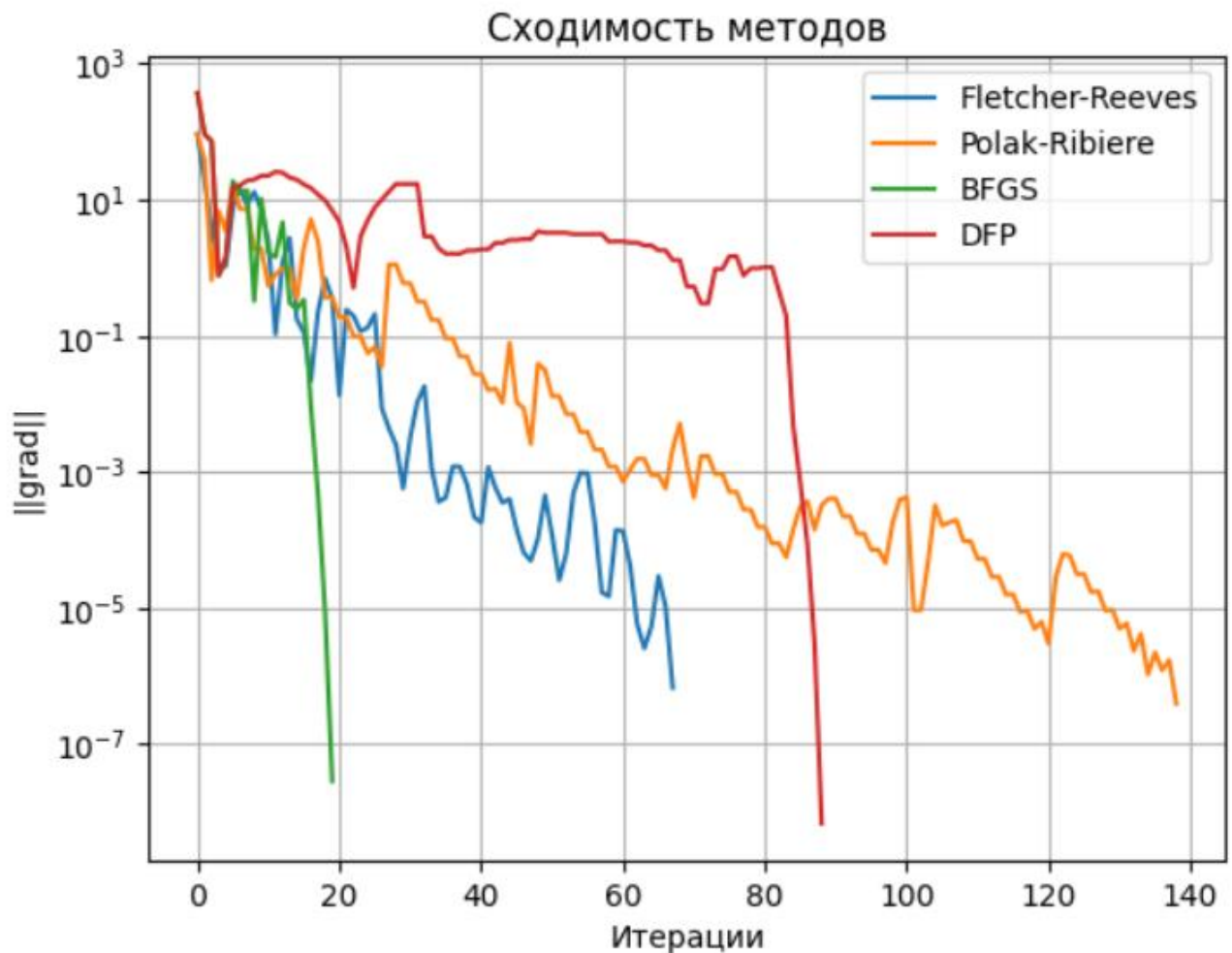


Рис. 1 – Графики сходимости различных методов

Методы сопряженных градиентов используют только информацию о градиенте функции, что делает их менее эффективными на функциях с сильно искривленной геометрией, таких как функция Розенброка

Квазиньютоновские методы (особенно BFGS) используют аппроксимацию матрицы Гессе, что позволяет учитывать кривизну функции и значительно ускоряет сходимость.

В ходе эксперимента метод BFGS показал наилучшую скорость сходимости как по числу итераций, так и по времени выполнения.

Вывод: были выполнены все задачи по плану данной лабораторной работы. Соответствующие действия по выполнению задач лабораторной работы были описаны и внесены в настоящий отчет. Были изучены алгоритмы многомерного поиска 1-го и 2-го порядка. Была разработана программа реализации алгоритмов многомерного поиска. Были вычислены экстремумы функции.

ПРИЛОЖЕНИЕ

Код программы с тестированием алгоритмов


```

import numpy as np

a, b, f0 = 158, 2, 40

def f(x):
    x1, x2 = x
    return a*(x1**2 - x2)**2 + b*(x1 - 1)**2 + f0

def grad(x):
    x1, x2 = x
    df_dx1 = 4*a*x1*(x1**2 - x2) + 2*b*(x1 - 1)
    df_dx2 = -2*a*(x1**2 - x2)
    return np.array([df_dx1, df_dx2])

```

Сопряженные градиенты (Fletcher-Reeves)

```

def conjugate_gradient_fr(x0, tol=1e-6, max_iter=1000):
    x = x0.copy()
    g = grad(x)
    d = -g

    for k in range(max_iter):
        alpha = line_search(x, d)
        x_new = x + alpha * d
        g_new = grad(x_new)

        if np.linalg.norm(g_new) < tol:
            break

        beta = np.dot(g_new, g_new) / np.dot(g, g)
        d = -g_new + beta * d

        x, g = x_new, g_new

    return x

```

Polak-Ribiere

```

def conjugate_gradient_pr(x0, tol=1e-6, max_iter=1000):
    x = x0.copy()
    g = grad(x)

```

```

d = -g

for k in range(max_iter):
    alpha = line_search(x, d)
    x_new = x + alpha * d
    g_new = grad(x_new)

    if np.linalg.norm(g_new) < tol:
        break

    beta = np.dot(g_new, g_new - g) / np.dot(g, g)
    d = -g_new + beta * d

    x, g = x_new, g_new

return x

```

Квазиньютоновские методы (BFGS)

```

def bfgs(x0, tol=1e-6, max_iter=1000):
    x = x0.copy()
    n = len(x)
    H = np.eye(n)

    for _ in range(max_iter):
        g = grad(x)
        if np.linalg.norm(g) < tol:
            break

        d = -H @ g
        alpha = line_search(x, d)

        s = alpha * d
        x_new = x + s
        y = grad(x_new) - g

        rho = 1.0 / (y @ s)
        I = np.eye(n)

        H = (I - rho * np.outer(s, y)) @ H @ (I - rho * np.outer(y, s)) + rho * np.outer(s, s)

        x = x_new

```

```
return x
```

DFP

```
def dfp(x0, tol=1e-6, max_iter=1000):
    x = x0.copy()
    n = len(x)
    H = np.eye(n)

    for _ in range(max_iter):
        g = grad(x)
        if np.linalg.norm(g) < tol:
            break

        d = -H @ g
        alpha = line_search(x, d)

        s = alpha * d
        x_new = x + s
        y = grad(x_new) - g

        H += np.outer(s, s)/(s@y) - (H @ np.outer(y, y) @ H)/(y @ H @ y)

        x = x_new

    return x
```

L-BFGS

```
from scipy.optimize import minimize

res = minimize(f, x0=[-1, 2], method='L-BFGS-B', jac=grad)
print(res.x)

[1.00000011 1.00000001]
```

Line-Search

```
def line_search(x, d, alpha=1.0, rho=0.5, c=1e-4):
    while f(x + alpha*d) > f(x) + c*alpha*np.dot(grad(x), d):
```

```
    alpha *= rho
    return alpha
```

Execution Time Estimation

```
import time

def test(method, x0):
    start = time.time()
    x = method(x0)
    t = time.time() - start
    return x, f(x), t
```

Результаты

```
import pandas as pd

# начальная точка (типично для Розенброка)
x0 = np.array([-1.2, 1.0])

# --- обёртки для подсчёта итераций ---

def run_with_stats(method, name):
    history = []

    def wrapped_method(x0):
        x = x0.copy()
        g = grad(x)
        d = -g

        start = time.time()
        iters = 0

        # разные методы имеют разную логику → проверяем по имени
        if name in ["FR", "PR"]:
            for _ in range(1000):
                alpha = line_search(x, d)
                x_new = x + alpha * d
                g_new = grad(x_new)

                history.append(np.linalg.norm(g_new))
```

```

    iters += 1

    if np.linalg.norm(g_new) < 1e-6:
        break

    if name == "FR":
        beta = np.dot(g_new, g_new) / np.dot(g, g)
    else: # PR
        beta = np.dot(g_new, g_new - g) / np.dot(g, g)

    d = -g_new + beta * d
    x, g = x_new, g_new

elif name in ["BFGS", "DFP"]:
    n = len(x)
    H = np.eye(n)

    for _ in range(1000):
        g = grad(x)
        history.append(np.linalg.norm(g))
        iters += 1

        if np.linalg.norm(g) < 1e-6:
            break

        d = -H @ g
        alpha = line_search(x, d)

        s = alpha * d
        x_new = x + s
        y = grad(x_new) - g

        if name == "BFGS":
            rho = 1.0 / (y @ s)
            I = np.eye(n)
            H = (I - rho * np.outer(s, y)) @ H @ (I - rho * np.outer(y, s)) + rho * np.outer(s, s)
        else: # DFP
            H += np.outer(s, s) / (s @ y) - (H @ np.outer(y, y) @ H) / (y @ H @ y)

        x = x_new

elapsed = time.time() - start

```

```

        return x, f(x), iters, elapsed, history

    return wrapped_method(x0)

# --- запуск всех методов ---

results = []

methods = [
    ("Fletcher-Reeves", "FR"),
    ("Polak-Ribiere", "PR"),
    ("BFGS", "BFGS"),
    ("DFP", "DFP"),
]

for name_full, name_short in methods:
    x_min, f_min, iters, t, history = run_with_stats(None, name_short)

    results.append({
        "Метод": name_full,
        "x": x_min,
        "f(x)": f_min,
        "Итерации": iters,
        "Время (сек)": t
    })

# L-BFGS отдельно через scipy
from scipy.optimize import minimize

start = time.time()
res = minimize(f, x0, method='L-BFGS-B', jac=grad)
elapsed = time.time() - start

results.append({
    "Метод": "L-BFGS",
    "x": res.x,
    "f(x)": res.fun,
    "Итерации": res.nit,
    "Время (сек)": elapsed
})

```

```
# --- ВЫВОД ТАБЛИЦЫ ---
df = pd.DataFrame(results)
print(df)
```

	Метод	x	f(x)	Итерации	\
0	Fletcher-Reeves	[1.0000000037633409, 1.00000000606340727]	40.0	68	
1	Polak-Ribiere	[1.00000000888378342, 1.00000001805469871]	40.0	139	
2	BFGS	[1.00000000001097906, 1.00000000002605771]	40.0	20	
3	DFP	[1.00000000344212259, 1.00000000693324473]	40.0	910	
4	L-BFGS	[0.99999999563490786, 0.99999999276403196]	40.0	35	

	Время (сек)
0	0.007428
1	0.024617
2	0.001389
3	0.148387
4	0.014062

```
import matplotlib.pyplot as plt
```

```
for name_full, name_short in methods:
    _, _, _, history = run_with_stats(None, name_short)
    plt.plot(history, label=name_full)
```

```
plt.yscale("log")
plt.xlabel("Итерации")
plt.ylabel("||grad||")
plt.title("Сходимость методов")
plt.legend()
plt.grid()
plt.show()
```

